

FASE 2 — Auditoría del código

Tabla de Hallazgos — AuthController.java

#	Descripción del problema	Archivo	Línea aprox.	Principio violado	Riesgo
1	Uso de nombres de parámetros poco descriptivos (u, p, e) que afectan legibilidad y claridad del código	AuthController.java	20–28	Clean Code (Naming claro y descriptivo)	Bajo
2	Uso de throws Exception genérico en los endpoints, sin manejo específico de errores	AuthController.java	20, 26	Clean Code (Manejo adecuado de excepciones)	Medio
3	Retorno de Map<String, Object> en lugar de un DTO tipado, lo que debilita el contrato de la API	AuthController.java	19–29	Clean Architecture / Buen diseño de API	Medio
4	Ausencia de validación de parámetros de entrada (@Valid, @NotBlank, @Email, etc.) en el controlador	AuthController.java	20–28	Seguridad básica / Separación de responsabilidades	Medio

Tabla de Hallazgos — AuthService.java

#	Descripción del problema	Archivo	Línea aprox.	Principio violado	Riesgo
1	Uso de algoritmo MD5 para hash de contraseñas (inseguro y obsoleto)	AuthService.java	56–67	Seguridad básica (Hashing inseguro)	Alto
2	Retorno del hash de la contraseña en la respuesta del login	AuthService.java	27, 32	Exposición de información sensible	Alto
3	Acceso directo a atributos públicos (c.password, c.email, c.username)	AuthService.java	23–28, 44–46	Encapsulamiento (Clean Code / OOP)	Medio
4	Logging mediante System.out.println exponiendo información sensible	AuthService.java	24–31, 47–51	Seguridad / Buenas prácticas de logging	Alto

FASE 4 — ADR

ADR-001: Refactorización del módulo de autenticación por vulnerabilidades de seguridad y malas prácticas de Diseño

El sistema actual permite registro y autenticación de usuarios mediante un módulo compuesto por controller, service y repository. Durante la auditoría se identificaron problemas críticos: uso de MD5 para contraseñas, exposición del hash en las respuestas del login, validación débil de contraseñas y logging inseguro con `System.out.println`. Además, se detectaron violaciones al principio de responsabilidad única y uso de estructuras genéricas en las respuestas.

Estas fallas representan un riesgo alto en producción, ya que exponen información sensible y facilitan ataques de fuerza bruta. Esto afecta directamente a los usuarios (seguridad), al equipo técnico (mantenibilidad) y al negocio (riesgo reputacional).

Decisión

1. Se reemplazará MD5 por BCrypt para el almacenamiento seguro de contraseñas.
2. Se eliminará el campo hash de las respuestas y se implementarán DTOs tipados.
3. Se reforzará la validación de contraseñas (mínimo 8 caracteres y mayor complejidad).
4. Se reemplazará `System.out.println` por logging estructurado.
5. Se refactorizará el servicio aplicando SRP para separar validación, autenticación y construcción de respuesta.

La arquitectura quedará con responsabilidades claras: Controller (entrada), Service (lógica), Repository (persistencia) y DTOs (contratos).

Consecuencias

Positivas

- Mayor seguridad.
- Eliminación de exposición de datos sensibles.
- Código más limpio y mantenible.
- Mejor cumplimiento de buenas prácticas.

Riesgos

- Tiempo adicional de refactorización.
- Posibles regresiones si no se agregan pruebas.
- Ajustes necesarios en contratos de API.

Alternativas consideradas

1. Reescribir el módulo completo → descartado por alto costo y tiempo innecesario.
2. Corregir solo el hashing sin refactorizar estructura → descartado porque no solucionaría problemas de diseño ni exposición de datos.